

# Test Cases — Connection & UI Fixes

## Testdokumentation für Flutter App (ConnectionManager & UI)

Datum: 15.05.2026

Tester: Jonathan Schmidt

Version: 1.1

Testumgebung: Flutter Web &amp; Android

### TC-013: ConnectionStatus zeigt falsch "Connected"

Hoch

Funktional

Äquivalenzklasse: Keine Verbindung (Verbindungszustände / Keine Verbindung zum Fahrzeug möglich)

Verbindungszustände

#### BESCHREIBUNG

Test ob der ConnectionStatus-Button den korrekten Verbindungsstatus anzeigt.

#### VORBEDINGUNGEN

- App gestartet
- ESP32 Auto ausgeschaltet (keine Verbindung möglich)

#### TESTSCHRITTE

1. App öffnen
2. ConnectionStatus-Button beobachten
3. Verbindungsversuch zum Auto starten
4. Status-Farbe prüfen (sollte rot/grau bleiben)
5. ESP32 einschalten
6. Erneut verbinden
7. Status-Farbe prüfen (sollte grün werden)

#### ERWARTETES ERGEBNIS

- Button zeigt **rot/grau** wenn keine Verbindung
- Button zeigt **grün** nur bei erfolgreicher WebSocket-Verbindung
- Status ändert sich sofort bei Verbindungsaufbau

#### TATSÄCHLICHES ERGEBNIS (VOR FIX)

##### FEHLGESCHLAGEN

- Button zeigte **grün** obwohl keine Verbindung bestand
- `connected.value = true` wurde gesetzt ohne echten Handshake
- `Blind await Future.delayed(100ms)` wartete nur Zeit ab
- Befehle konnten nicht gesendet werden trotz grünem Status

#### TATSÄCHLICHES ERGEBNIS (NACH FIX)

##### BESTANDEN

- Button zeigt korrekt rot/grau ohne Verbindung
- Button zeigt grün nur nach erfolgreichem WebSocket-Handshake
- `await _channel!.ready` wartet auf echte Verbindung
- Status ist zuverlässig

## ROOT CAUSE

```
// Bug: Blind delay, kein echter Handshake-Check
Future<void> _connect() async {
  _channel = WebSocketChannel.connect(Uri.parse(wsUrl));
  await Future.delayed(Duration(milliseconds: 100)); // Blind!
  connected.value = true; // Wird immer gesetzt
}
```

## FIX

```
// Fix: Warten auf echten WebSocket-Handshake
Future<void> _connect() async {
  _channel = WebSocketChannel.connect(Uri.parse(wsUrl));
  await _channel!.ready; // Wartet auf Handshake

  connected.value = true; // Nur bei Erfolg
  _reconnectSeconds = 1;
  _startKeepAlive();
  // ...
}
```

## VERIFIKATION

- Status korrekt bei Offline
- Status korrekt bei Online
- Keine False-Positives mehr
- Befehle nur bei echter Verbindung

## TC-014: Verbindung trennt sich automatisch

**Kritisch****Funktional****Äquivalenzklasse: Instabile Verbindung (Verbindungszustände / Wiederholte Verbindungsabbrüche)**

Verbindungszustände

## BESCHREIBUNG

Test der Verbindungsstabilität über längere Zeit.

## VORBEDINGUNGEN

- App gestartet
- Verbindung zum Auto hergestellt

- Auto eingeschaltet

## TESTSCHRITTE

1. Verbindung zum Auto herstellen
2. ConnectionStatus prüfen (sollte grün sein)
3. 30 Sekunden warten ohne Befehle zu senden
4. ConnectionStatus erneut prüfen
5. Befehl senden (z.B. "forward")
6. Reaktion des Autos beobachten

## ERWARTETES ERGEBNIS

- Verbindung bleibt **stabil** über 30+ Sekunden
- Status bleibt **grün**
- Befehle werden nach Wartezeit noch empfangen
- Keine automatischen Disconnects

## TATSÄCHLICHES ERGEBNIS (VOR FIX)

### FEHLGESCHLAGEN

- Verbindung wechselte alle paar Sekunden auf **Offline**
- Status: Grün → Rot → Grün → Rot (zyklisch)
- ESP32 trennte idle Verbindungen automatisch
- Befehle gingen verloren
- Ständige Reconnect-Versuche

## TATSÄCHLICHES ERGEBNIS (NACH FIX)

### BESTANDEN

- Verbindung bleibt stabil über 5+ Minuten
- Status bleibt konstant **grün**
- Keepalive-Ping alle 5 Sekunden verhindert Timeout
- Keine ungewollten Disconnects
- Befehle werden zuverlässig empfangen

## ROOT CAUSE

- ESP32 trennt WebSocket-Verbindungen nach Inaktivität
- Kein Keepalive-Mechanismus implementiert
- Idle-Timeout des ESP32 nicht berücksichtigt

## FIX

```
void _startKeepAlive() {
    _keepAliveTimer?.cancel();
    _keepAliveTimer = Timer.periodic(const Duration(seconds: 5), (_) {
        if (_channel != null && connected.value) {
            try {
                _channel!.sink.add('ping'); // Keepalive-Signal
            } catch (e) {
```

```
        print('Keepalive failed: $e');
    }
}
});
}

void _stopKeepAlive() {
    _keepAliveTimer?.cancel();
    _keepAliveTimer = null;
}

// In _connect():
await _channel!.ready;
connected.value = true;
_startKeepAlive(); // Keepalive starten
```

## VERIFIKATION

- Verbindung stabil über 5+ Minuten
- Ping alle 5 Sekunden gesendet
- ESP32 trennt nicht mehr
- Timer wird bei Disconnect gestoppt
- Keine Ressourcen-Leaks

## METRIKEN

| Metrik              | Vorher     | Nachher |
|---------------------|------------|---------|
| Verbindungsdauer    | ~10-20 sec | 5+ min  |
| Disconnects/min     | 3-6        | 0       |
| Keepalive-Intervall | —          | 5 sec   |
| Befehls-Erfolgsrate | ~60%       | ~99%    |

## TC-015: WASD-Geschwindigkeitsanzeige aktualisiert nicht

**Mittel****UI/UX****Äquivalenzklasse: Sofortige Befehlsausführung (Eingabemethoden / Tastatureingaben & Touch-Eingaben)****Befehlsverarbeitung**

## BESCHREIBUNG

Test der Geschwindigkeitsanzeige bei Tastatur-Steuerung (WASD).

## VORBEDINGUNGEN

- App gestartet
- Driving Mode aktiv

- Tastatur verfügbar (Browser oder Desktop)

## TESTSCHRITTE

1. Taste "W" drücken und halten (3 Sekunden)
2. Geschwindigkeitsanzeige im HUD beobachten
3. Taste loslassen
4. Geschwindigkeit sollte auf 0 zurückgehen
5. Gas-Button (On-Screen) drücken und halten
6. Geschwindigkeitsanzeige erneut beobachten
7. Vergleichen: WASD vs. On-Screen-Buttons

## ERWARTETES ERGEBNIS

- Geschwindigkeitsanzeige aktualisiert sich bei **WASD-Eingabe**
- Geschwindigkeitsanzeige aktualisiert sich bei **Button-Eingabe**
- Beide Eingabemethoden zeigen identisches Verhalten
- HUD zeigt korrekte Geschwindigkeit in Echtzeit

## TATSÄCHLICHES ERGEBNIS (VOR FIX)

### FEHLGESCHLAGEN

- Geschwindigkeitsanzeige aktualisierte sich **nur** bei On-Screen-Buttons
- Bei WASD-Eingabe: HUD zeigte **0 km/h** trotz Fahrt
- Auto fuhr (WebSocket-Befehle wurden gesendet)
- Aber lokaler State wurde nicht aktualisiert
- Physikschleife `_updatePhysics()` hatte keine Daten

## TATSÄCHLICHES ERGEBNIS (NACH FIX)

### BESTANDEN

- Geschwindigkeitsanzeige aktualisiert sich bei WASD
- Geschwindigkeitsanzeige aktualisiert sich bei Buttons
- Identisches Verhalten für beide Eingabemethoden
- HUD zeigt korrekte Geschwindigkeit
- Physikschleife funktioniert mit beiden Inputs

## ROOT CAUSE

```
// Bug: Globaler handleKey-Handler
void handleKey(RawKeyEvent event) {
    // ...
    if (key == LogicalKeyboardKey.arrowUp) {
        isDown ? activeCommands.add('forward') : activeCommands.remove('forward');
    }
    updateCommand(); // Sendet WebSocket-Befehl

    // ABER: Setzt nie accelerating, braking, steerLeft, steerRight
    // → _updatePhysics() hat keine Daten
    // → HUD zeigt 0 km/h
}
```

**Problem:** Globaler Handler aktualisierte nur `activeCommands` (für WebSocket), nicht die lokalen State-Variablen (`accelerating`, `braking`, etc.), die `_updatePhysics()` benötigt.

## FIX

```
// Neue Methode in _DrivingPageState
void _handleDrivingKey(RawKeyEvent event) {
  final key = event.logicalKey;
  final isDown = event is RawKeyDownEvent;
  if (keyPressed[key] == isDown) return;
  keyPressed[key] = isDown;

  // Ruft dieselben Methoden auf wie die Buttons
  if (key == LogicalKeyboardKey.arrowUp || key == LogicalKeyboardKey.keyW) {
    controlsInverted ? _inputBrake(isDown) : _inputAccelerate(isDown);
  } else if (key == LogicalKeyboardKey.arrowDown || key == LogicalKeyboardKey.keyS) {
    controlsInverted ? _inputAccelerate(isDown) : _inputBrake(isDown);
  } else if (key == LogicalKeyboardKey.arrowLeft || key == LogicalKeyboardKey.keyA) {
    _inputSteerLeft(isDown);
  } else if (key == LogicalKeyboardKey.arrowRight || key == LogicalKeyboardKey.keyD) {
    _inputSteerRight(isDown);
  }
}

// Eingehängt als onKey-Handler
RawKeyboardListener(
  focusNode: _focusNode,
  onKey: _handleDrivingKey, // Neue Methode
  child: ...
)
```

## VERIFIKATION

- WASD aktualisiert HUD
- Buttons aktualisieren HUD
- Identisches Verhalten
- Physikschleife funktioniert
- Geschwindigkeit korrekt angezeigt

## TC-016: Sensor-Popup für Autonomes Fahren

**Mittel****Funktional**

Äquivalenzklasse: Sensorprüfung nicht durchgeführt (Systeminitialisierung / Autonomer Modus ohne durchgeführte Sensorabfrage)

**Systeminitialisierung**

## BESCHREIBUNG

Test des Sensor-Prüfungs-Dialogs beim Verbindungsaufbau.

### VORBEDINGUNGEN

- App gestartet
- Autonomous Driving Page geöffnet
- Auto ausgeschaltet (Offline)

### TESTSCHRITTE

1. Autonomous Driving Page öffnen (Status: Offline)
2. Auto einschalten
3. Verbindung herstellen (Status: Offline → Online)
4. Dialog beobachten
5. "Ja" auswählen
6. START AUTONOMOUS MODE Button prüfen (sollte aktiviert sein)
7. Verbindung trennen (Auto ausschalten)
8. Auto wieder einschalten
9. Dialog sollte erneut erscheinen

### ERWARTETES ERGEBNIS

- Dialog erscheint **nur** auf Autonomous Driving Page
- Dialog erscheint bei **jedem** Verbindungsaufbau
- "Ja" → sendet `sensor0n` an ESP32
- "Nein" → kein Befehl
- START-Button deaktiviert bis Dialog beantwortet
- Bei Disconnect wird Dialog-Status zurückgesetzt

### TATSÄCHLICHES ERGEBNIS (VOR FIX)

#### NICHT IMPLEMENTIERT

- Kein Sensor-Dialog vorhanden
- Sensor wurde nie aktiviert
- Autonomes Fahren funktionierte nicht
- Keine Möglichkeit `sensor0n` zu senden

### TATSÄCHLICHES ERGEBNIS (NACH FIX)

#### BESTANDEN

- Dialog erscheint bei Verbindungsaufbau
- Nur auf Autonomous Driving Page
- "Ja" sendet `sensor0n` erfolgreich
- "Nein" sendet nichts
- START-Button korrekt deaktiviert/aktiviert
- Dialog-Status wird bei Disconnect zurückgesetzt
- Dialog erscheint bei erneutem Connect

### IMPLEMENTIERUNG

```

class _AutonomousDrivingPageState extends State<AutonomousDrivingPage> {
  bool _sensorCheckDone = false;

  @override
  void initState() {
    super.initState();
    ConnectionManager.instance.connected.addListener(_onConnectionChanged);
    if (ConnectionManager.instance.connected.value) {
      WidgetsBinding.instance.addPostFrameCallback(_) {
        if (mounted) _showSensorPopup();
      };
    }
  }

  void _onConnectionChanged() {
    if (!mounted) return;
    if (ConnectionManager.instance.connected.value) {
      // Verbindung hergestellt → Dialog zeigen
      if (!_sensorCheckDone) {
        WidgetsBinding.instance.addPostFrameCallback(_) {
          if (mounted) _showSensorPopup();
        };
      }
    } else {
      // Verbindung verloren → Status zurücksetzen
      setState(() { _sensorCheckDone = false; isRunning = false; });
    }
  }

  Future<void> _showSensorPopup() async {
    final result = await showDialog<bool>(
      context: context,
      barrierDismissible: false,
      builder: (ctx) => AlertDialog(
        title: const Text('Sensor-Prüfung'),
        content: const Text('Ist ein Sensor im Auto verbaut?'),
        actions: [
          TextButton(onPressed: () => Navigator.pop(ctx, false), child: const Text('Nein')),
          ElevatedButton(onPressed: () => Navigator.pop(ctx, true), child: const Text('Ja')),
        ],
      ),
    );
    if (!mounted) return;
    setState(() => _sensorCheckDone = true);
    if (result == true) { ConnectionManager.instance.send('sensorOn'); }
  }

  void _startAutonomous() {
    if (!_sensorCheckDone) return; // Guard
    setState(() => isRunning = true);
    ConnectionManager.instance.send('auto');
  }
}

```

## VERIFIKATION



- Dialog erscheint bei Connect
- Nur auf Autonomous Page
- "Ja" sendet sensor0n
- "Nein" sendet nichts
- START-Button Guard funktioniert
- Status-Reset bei Disconnect
- Dialog erscheint erneut bei Reconnect

## USER FLOW

1. User öffnet Autonomous Driving Page
  - ↓
2. Status: Offline
  - ↓
3. Auto einschalten
  - ↓
4. Verbindung hergestellt (Offline → Online)
  - ↓
5. Dialog erscheint: "Ist ein Sensor im Auto verbaut?"
  - ↓
- 6a. User wählt "Ja"
  - ↓
  - sensor0n gesendet
  - ↓
- 6b. User wählt "Nein"
  - ↓
  - Kein Befehl
  - ↓
7. \_sensorCheckDone = true
  - ↓
8. START AUTONOMOUS MODE aktiviert
  - ↓
9. User kann autonomes Fahren starten

## TC-017: Token-Bucket-Throttling entfernt

Hoch

Performance

Äquivalenzklasse: Sofortige Befehlsausführung (Befehlsverarbeitung / Befehle ohne Verzögerung an das Fahrzeug gesendet)

Befehlsverarbeitung

### BESCHREIBUNG

Verifikation dass Token-Bucket-Throttling entfernt wurde und Befehle wieder direkt gesendet werden.

### VORBEDINGUNGEN

- App gestartet
- Driving Mode aktiv
- Verbindung zum Auto hergestellt

### TESTSCHRITTE

1. Gas-Button drücken und halten (5 Sekunden)
2. Fahrt beobachten (sollte flüssig sein)
3. Schnell zwischen Links/Rechts wechseln (5× pro Sekunde)
4. Lenkverhalten beobachten
5. Gas loslassen
6. Stop-Befehl-Verzögerung messen

#### ERWARTETES ERGEBNIS

- Befehle werden **sofort** gesendet
- Keine künstliche Verzögerung
- Flüssige Fahrt ohne Ruckeln
- Stop-Befehl kommt sofort
- Responsive Steuerung

#### TATSÄCHLICHES ERGEBNIS (MIT TOKEN-BUCKET)

##### FEHLGESCHLAGEN

- Befehle wurden verzögert (Token-Bucket-Limit)
- Ruckelige Fahrt
- Stop-Befehle verzögert (bis zu 200ms)
- Steuerung unbrauchbar
- Schlechte User Experience

#### TATSÄCHLICHES ERGEBNIS (NACH ENTFERNUNG)

##### BESTANDEN

- Befehle werden sofort gesendet
- Flüssige, responsive Fahrt
- Stop-Befehle ohne Verzögerung
- Steuerung präzise und brauchbar
- Gute User Experience

#### BEGRÜNDUNG FÜR ENTFERNUNG

##### Token-Bucket wurde bewusst entfernt weil:

1. **Ruckeln:** Künstliche Verzögerungen machten Fahrt ruckelig
2. **Stop-Verzögerung:** Sicherheitskritisch — Stop muss sofort kommen
3. **Unbrauchbar:** Steuerung war nicht mehr präzise nutzbar
4. **ESP32 kann es:** ESP32 verarbeitet Commands schnell genug
5. **WebSocket-Overhead:** WebSocket ist bereits effizient

#### CODE-ÄNDERUNG

```
// Vorher: Token-Bucket-Throttling
void send(String cmd) {
    if (_tokens <= 0) {
        _pendingCommand = cmd; // Verzögert
        return;
    }
    // ...
}
```

```
}
_tokens--;
_channel?.sink.add(cmd);
}

// Nachher: Direkt senden
void send(String cmd) {
    if (_channel != null && connected.value) {
        try {
            _channel!.sink.add(cmd); // Sofort
        } catch (e) {
            print('Send failed: $e');
        }
    }
}
```

## VERIFIKATION

- Kein Token-Bucket mehr
- Befehle sofort gesendet
- Keine künstlichen Delays
- Flüssige Steuerung
- Stop-Befehle sofort

## Test-Zusammenfassung

### STATISTIK

- **Gesamt Tests:** 5 (TC-013 bis TC-017)
- **Kritisch:** 1
- **Hoch:** 2
- **Mittel:** 2

### ERGEBNISSE (VOR FIXES)

- **Fehlgeschlagen:** 4
- **Nicht implementiert:** 1

### ERGEBNISSE (NACH FIXES)

- **Bestanden:** 5
- **Fehlgeschlagen:** 0

### KATEGORIEN

- Funktional: 3 Tests
- UI/UX: 1 Test
- Performance: 1 Test

### KRITISCHE FIXES

1. **Verbindungsstabilität** (TC-014): Keepalive verhindert Disconnects
2. **ConnectionStatus** (TC-013): Echter Handshake-Check
3. **Token-Bucket entfernt** (TC-017): Responsive Steuerung

## Lessons Learned

1. **WebSocket-Handshake:** `await channel.ready` ist essentiell für zuverlässigen Status
2. **Keepalive wichtig:** ESP32 trennt idle Verbindungen — Ping alle 5s verhindert Timeout
3. **State-Synchronisation:** Tastatur-Handler müssen dieselben State-Methoden aufrufen wie UI-Buttons
4. **Sensor-Dialog:** User-Feedback vor kritischen Operationen (autonomes Fahren) einholen
5. **Throttling-Trade-off:** Manchmal ist "zu viel Optimierung" schlechter als direkte Implementierung

## Änderungshistorie

| Datum      | Version | Änderung                                     |
|------------|---------|----------------------------------------------|
| 15.06.2026 | 1.0     | Initiale Test-Dokumentation                  |
| 15.06.2026 | 1.1     | TC-017 hinzugefügt (Token-Bucket-Entfernung) |

**Dokumentiert von:** Alexa van der Meulen | **Review:** Team R.E.D. | **Status:** Alle Tests bestanden